



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Organización y Arquitectura de Computadoras

Tarea 2. Sumatoria de productos (producto punto) en ensamblador

Grupo 1

Ortega de la Paz Rafael 420054085

19 de octubre de 2026

Objetivo

Programar en lenguaje ensamblador, con la unidad de punto flotante de la Raspberry Pi Pico 2, la sumatoria de productos de dos vectores de igual dimensión (producto punto), recorriendo la memoria mediante apuntadores que avanzan de 4 en 4 bytes.

Introducción

La Raspberry Pi Pico 2 utiliza el microcontrolador RP2350, que integra dos núcleos Arm Cortex-M33 a 150 MHz. A diferencia de la Pico original (RP2040, núcleos Cortex-M0+), cada núcleo Cortex-M33 incluye una unidad de punto flotante (FPU) de precisión simple con 32 registros de 32 bits, S0 a S31, que ejecuta en hardware la suma, la resta, la multiplicación, la división y la raíz cuadrada. La FPU no ofrece funciones trascendentes como seno, coseno o logaritmo; esas deben construirse a partir de las operaciones básicas.

MicroPython permite escribir funciones en ensamblador Thumb dentro de un programa en Python mediante el decorador *@micropython.asm_thumb*. Los primeros cuatro argumentos de la función llegan en los registros r0 a r3 y el valor de regreso se toma de r0. Cuando el argumento es un arreglo (*array*), el registro recibe la dirección de su primer elemento, de modo que el ensamblador puede leer y escribir la memoria del arreglo con instrucciones de carga y almacenamiento (ldr/str para enteros, vldr/vstr para flotantes). Como la memoria se direcciona por bytes y cada dato de 32 bits ocupa cuatro, el desplazamiento del elemento i es $4 \cdot i$.

El producto punto de dos vectores **a** y **b** de N elementos se define como:

$$a \cdot b = a_0 b_0 + a_1 b_1 + \ldots + a_n b_n = \Sigma\, a_i b_i, \quad \text{con } n = N - 1$$

Esta operación de multiplicar y acumular es la base de los filtros digitales, de la multiplicación de matrices y de muchos algoritmos de procesamiento de señales, por lo que sirve como preparación para el filtro digital del segundo proyecto.

Desarrollo

La rutina *producto_punto* recibe cuatro argumentos: en r0 y r1 las direcciones de los dos vectores, en r2 la dirección del arreglo donde se guarda el resultado y en r3 el número de elementos. Ambos vectores se declaran en MicroPython como arreglos de flotantes de 32 bits, *array('f', …)*. La Tabla 1 resume los registros.

Tabla 1. Registros utilizados en la rutina.

Registro	Contenido
r0, r1	Apuntadores a data1[i] y data2[i]
r2	Dirección del resultado
r3	Contador de elementos restantes
s0, s1	Elementos leídos de cada vector
s2	Producto data1[i]·data2[i]
s4	Acumulador de la suma

Antes del ciclo, el acumulador s4 se pone en cero copiando a la FPU un registro entero con valor 0 (*vmov*), ya que el patrón de 32 bits en cero corresponde a 0.0 en formato IEEE 754. Si *N* es 0 o negativo, la rutina salta directamente al final. En cada iteración se cargan los elementos con *vldr*, se multiplican con *vmul*, se acumulan con *vadd* y ambos apuntadores avanzan 4 bytes con *add*. Después se decrementa el contador con *sub*, que actualiza las banderas, y *bgt* repite el ciclo mientras queden elementos. Al terminar, *vstr* guarda s4 en el resultado.

Pruebas y resultados

Se probaron cuatro casos: el ejemplo de clase [1, 2]·[3, 4] = 11; un vector de cuatro elementos, [1, 2, 3, 4]·[2, 3, 4, 5] = 40; un caso con decimales y negativos, [0.5, −1.5, 2.25]·[4, 2, −1] = −3.25; y un ciclo largo de 100 elementos, [1, 2, …, 100]·[1, 1, …, 1] = 5050. Cada resultado se compara con el mismo cálculo hecho en Python. La Figura 1 muestra la salida en la Pico 2.



Figura 1. Salida del programa en la Pico 2: los cuatro casos coinciden con el cálculo en Python.

Los cuatro casos dieron exactamente el valor esperado. El caso de 100 elementos confirma que el contador y el avance de los apuntadores funcionan durante un ciclo largo. El programa incluye además un modo interactivo en el que se escriben dos vectores separados por comas y se obtiene su producto punto.

Conclusiones

Se implementó el producto punto en ensamblador usando la FPU de la Pico 2 y se obtuvieron resultados correctos en todos los casos de prueba. El ejercicio mostró en la práctica cómo recorrer arreglos en memoria: como la memoria se direcciona por bytes y cada flotante ocupa 32 bits, los apuntadores deben avanzar de 4 en 4. También se vio la importancia de inicializar el acumulador antes del ciclo y de controlar la repetición con un contador y un salto condicional. La operación de multiplicar y acumular desarrollada aquí es la misma que se usa en el filtro digital del segundo proyecto.

Bibliografía

Arm Limited. (2023). *Armv8-M architecture reference manual* (DDI0553). Arm Limited.

Arredondo Garza, J. A. de J. (2026). *Organización y Arquitectura de Computadoras: clases del semestre 2027-1* [Exposiciones en clase]. Facultad de Ingeniería, Universidad Nacional Autónoma de México.

IEEE. (2019). *IEEE standard for floating-point arithmetic* (IEEE Std 754-2019). Institute of Electrical and Electronics Engineers.

MicroPython. (s. f.). *Inline assembler for Thumb2 architectures*. Recuperado el 19 de octubre de 2026 de https://docs.micropython.org/en/latest/reference/asm_thumb2_index.html

Patterson, D. A., & Hennessy, J. L. (2016). *Computer organization and design: The hardware/software interface* (ARM ed.). Morgan Kaufmann.

Raspberry Pi Ltd. (2025). *RP2350 datasheet: A microcontroller by Raspberry Pi*. https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf

Anexo: código fuente (tarea_producto_punto.py)

```
# Tarea: Sumatoria de productos (producto punto) en ensamblador con la FPU
# Raspberry Pi Pico 2 (RP2350)
from array import array

# r0 -> data1 (array 'f'), r1 -> data2 (array 'f'), r2 -> res (array 'f', 1 elem), r3 -> N
@micropython.asm_thumb
def producto_punto(r0, r1, r2, r3):
    mov(r4, 0)
    vmov(s4, r4)          # acumulador s4 = 0.0 (bits en cero = 0.0 flotante)
    cmp(r3, 0)
    ble(FIN)              # N <= 0: no hay nada que sumar
    label(CICLO)
    vldr(s0, [r0, 0])     # s0 = data1[i]
    vldr(s1, [r1, 0])     # s1 = data2[i]
    vmul(s2, s0, s1)      # s2 = data1[i] * data2[i]
    vadd(s4, s4, s2)      # acumula
    add(r0, r0, 4)        # siguiente flotante: 4 bytes
    add(r1, r1, 4)
    sub(r3, r3, 1)        # N -= 1
    bgt(CICLO)            # repite mientras N > 0
    label(FIN)
    vstr(s4, [r2, 0])     # res = acumulador

def punto(a, b):
    if len(a) != len(b):
        raise ValueError("Los vectores deben tener la misma dimension")
    res = array('f', [0.0])
    producto_punto(a, b, res, len(a))
    return res[0]

# ---------- Pruebas ----------
casos = [
    ([1.0, 2.0], [3.0, 4.0]),                       # 11
    ([1.0, 2.0, 3.0, 4.0], [2.0, 3.0, 4.0, 5.0]),   # 40
    ([0.5, -1.5, 2.25], [4.0, 2.0, -1.0]),          # -3.25
    ([float(i) for i in range(1, 101)], [1.0] * 100),  # 5050
]
print("=== Producto punto en ensamblador (FPU) ===")
for a, b in casos:
    va, vb = array('f', a), array('f', b)
    r = punto(va, vb)
    ref = sum(x * y for x, y in zip(va, vb))
    print("N={:>3}  asm={:>12.4f}  python={:>12.4f}  {}".format(
        len(a), r, ref, "OK" if abs(r - ref) < 1e-3 * max(1, abs(ref)) else "ERROR"))

# ---------- Modo interactivo ----------
print()
def leer_vector(t):
    # acepta 1,2,3  (1,2,3)  [1, 2, 3]  o separados por espacios
    for c in "()[]":
        t = t.replace(c, " ")
    return [float(v) for v in t.replace(",", " ").split()]
```

```
print("=== Modo interactivo (Enter vacio para salir) ===")
while True:
    t1 = input("Vector 1 (ej. 1,2,3): ").strip()
    if not t1:
        break
    t2 = input("Vector 2 (misma dimension): ").strip()
    try:
        va = array('f', leer_vector(t1))
        vb = array('f', leer_vector(t2))
        print("  Producto punto (asm) = {:.4f}".format(punto(va, vb)))
    except ValueError as e:
        print("  Error:", e)
```